

# TABLA DE CONTENIDOS

- 01** Breve Historia
- 02** ¿Qué es la Ingeniería del Software?
- 03** **Ciclo de vida**
- 04** Herramientas CASE
- 05** Diagramas

# Ciclo de vida del Software

---

*¿Qué es el Ciclo de Vida del Software?*

*Visión Predictiva*

*Visión Adaptativa*

*La curva del coste del cambio*

# Ciclo de vida del Software

---

*¿Qué es el Ciclo de Vida del Software?*

*Visión Predictiva*

*Visión Adaptativa*

*La curva del coste del cambio*



# Ciclo de vida del Software

La ingeniería del software prescribe un conjunto de actividades ordenadas encaminadas a la producción de software. Estas actividades son conocidas como “ciclo de vida” del software:

1. **Requisitos.** Esta actividad tiene como objetivo descubrir los problemas o necesidades que el software debe solventar o suplir.
2. **Análisis.** Una vez aterrizadas las necesidades que debe suplir el software, en esta actividad se especificarán sus funcionalidades.
3. **Diseño Técnico.** Tras concretar las funcionalidades del software se diseña el sistema y se define su arquitectura.
4. **Codificación.** A partir del diseño y sobre los cimientos de la arquitectura propuesta se codifica el software.
5. **Pruebas** (Verificación y validación). El software debe funcionar correctamente y, además, responder a las expectativas del cliente.
6. **Despliegue.** Finalmente el software se despliega para comenzar a ser operado por los usuarios.

# Ciclo de vida del Software

| Actividad           | Objetivo                | Pregunta que trata de responder                                                                                                                          |
|---------------------|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Requisitos</b>   | Entender el problema    | ¿A qué problemas del cliente debe dar solución el software?                                                                                              |
| <b>Análisis</b>     | Esbozar la solución     | ¿Qué funcionalidad debe ofrecer el software para dar solución a los problemas descritos?                                                                 |
| <b>Diseño</b>       | Idear la solución       | ¿Cómo se implementarán los componentes del software que permitirán ofrecer la funcionalidad indicada?<br>¿Qué arquitectura soportará dichos componentes? |
| <b>Codificación</b> | Implementar la solución | ¿Cómo se desarrollará el código fuente de los componentes software propuestos?                                                                           |
| <b>Pruebas</b>      | Testar la solución      | ¿Se está construyendo el software correcto?<br>¿Se está construyendo correctamente el software?                                                          |
| <b>Despliegue</b>   | Usar la solución        | ¿Está el software en servicio adecuadamente?                                                                                                             |

# Ciclo de vida del Software

---

*¿Qué es el Ciclo de Vida del Software?*

*Visión Predictiva*

*Visión Adaptativa*

*La curva del coste del cambio*

*Metodologías*

*Modelos de Evaluación*

*Comparación entre visiones*



**VISIÓN PREDICTIVA**





**REQUISITOS**

**ANÁLISIS**

**DISEÑO**

**CODIFICACIÓN**

**PRUEBAS**

**DESPLIEGUE**





**REQUISITOS**

**ANÁLISIS**

**DISEÑO**

**CODIFICACIÓN**

**PRUEBAS**

**DESPLIEGUE**

TIEMPO

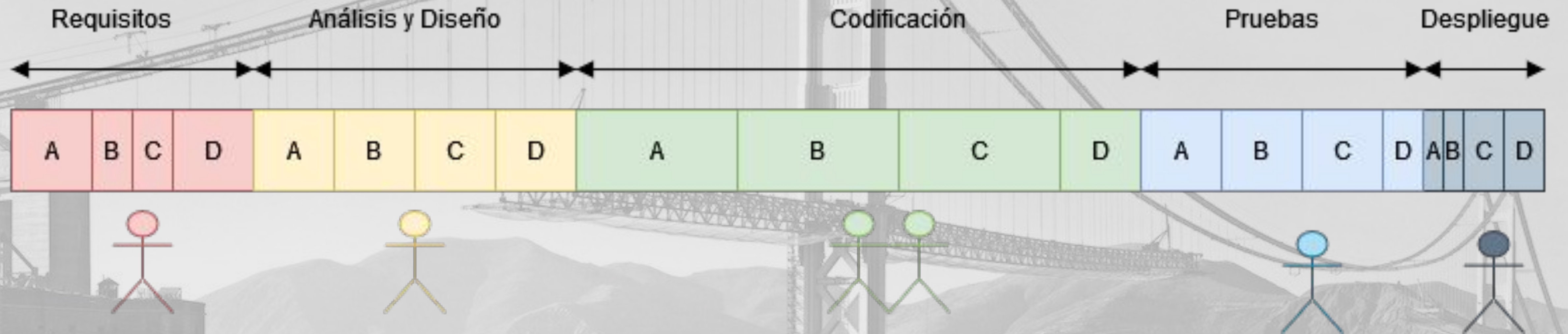


# VISIÓN PREDICTIVA





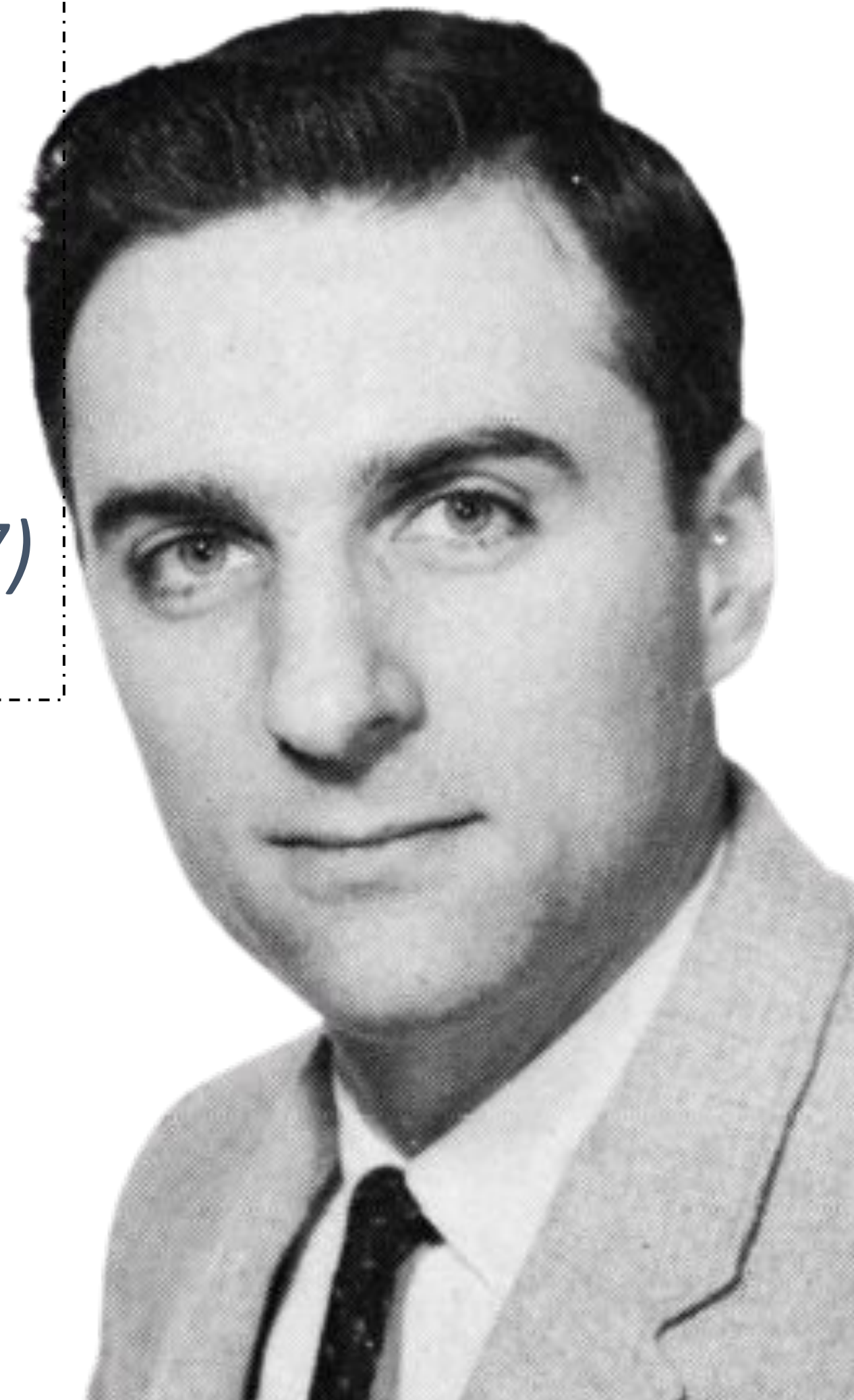
# VISIÓN PREDICTIVA





*“Las organizaciones dedicadas al diseño de sistemas [...] están abocadas a producir diseños que son copias de las estructuras de comunicación de dichas organizaciones ”*

*Melvin Conway, **Ley de Conway** (1967)*



# Ciclo de vida del Software

---

*¿Qué es el Ciclo de Vida del Software?*

*Visión Predictiva*

*Visión Adaptativa*

*La curva del coste del cambio*



# VISIÓN ADAPTATIVA







**¿Deben usarse el mismo método para construir un producto industrial que un producto software?**

**Hay cuatro características clave que hacen los productos software muy diferentes al productos industriales:**

- ❑ Coste de la materia prima**
- ❑ Maleabilidad del producto**
- ❑ Valor aportado por las personas**
- ❑ Factor de escala del producto**



# Coste de la materia prima

**Los recursos físicos necesarios son despreciables**



# Maleabilidad

**Si el software está bien construido es muy facil mantenerlo y extenderlo**



# Valor de las personas

**El grueso del conocimiento reside en el talento de las personas, no en los procedimientos**



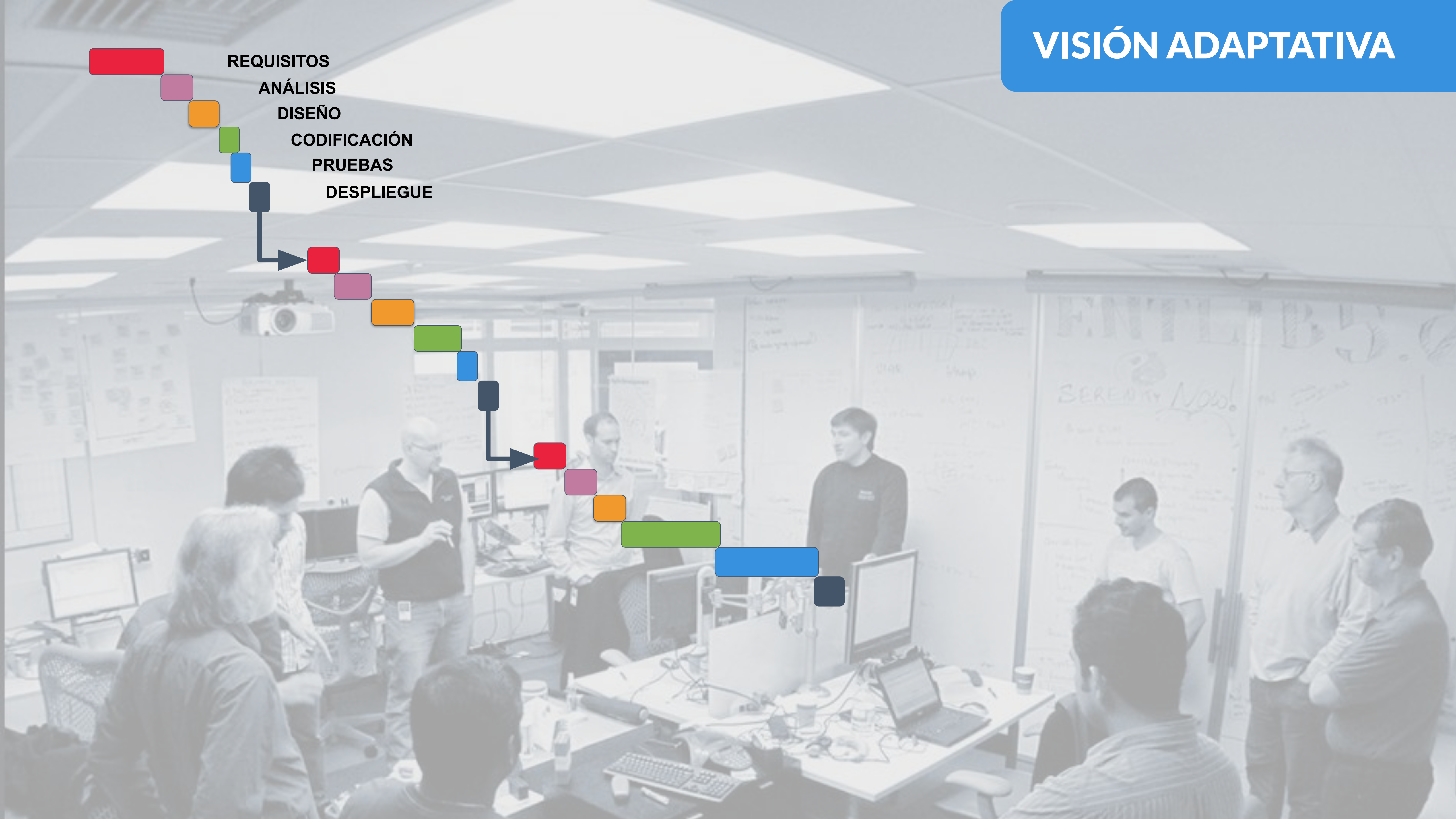
# Factor de Escala

**El factor de escala es infinito, cuesta lo mismo producir uno que mil**



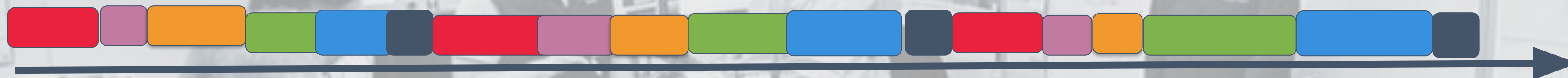


REQUISITOS  
ANÁLISIS  
DISEÑO  
CODIFICACIÓN  
PRUEBAS  
DESPLIEGUE





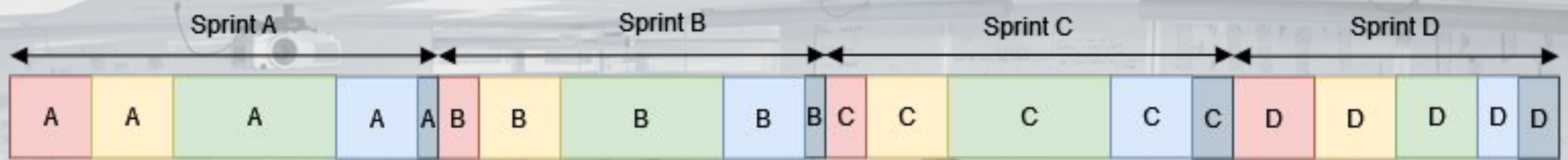
# VISIÓN ADAPTATIVA



TIEMPO



# VISIÓN ADAPTATIVA



TIEMPO



# VISIÓN ADAPTATIVA



TIEMPO



# VISIÓN ADAPTATIVA





# VISIÓN ADAPTATIVA



En marzo de 2001, 17 críticos de estos modelos, convocados por Kent Beck, que acababa de definir una nueva metodología denominada Extreme Programming, se reunieron en Salt Lake City para discutir sobre los modelos de desarrollo de software. El manifiesto ágil surgió con espíritu desafiante y beligerante contra los modelos predictivos.



# El Manifiesto Ágil

## VISIÓN ADAPTATIVA

*“Estamos poniendo al descubierto mejores métodos para desarrollar software, haciéndolo y ayudando a otros a que lo hagan. Con este trabajo hemos llegado a valorar:*

- *A los **individuos y su interacción**, por encima de los procesos y las herramientas.*
- *El **software que funciona**, por encima de la documentación exhaustiva.*
- *La **colaboración con el cliente**, por encima de la negociación contractual.*
- *La **respuesta al cambio**, por encima del seguimiento de un plan.*

*Aunque hay valor en los elementos de la derecha, valoramos más los de la izquierda.”*

- Kent Beck, Mike Beedle, Arie van Bennekum, Alistair Cockburn, Ward Cunningham, Martin Fowler, James Grenning, Jim Highsmith, Andrew Hunt, Ron Jeffries, Jon Kern, Brian Marick, Robert C. Martin, Steve Mellor, Ken Schwaber, Jeff Sutherland, Dave Thomas.



Tras los cuatro valores descritos, los firmantes redactaron los [12 principios](#) que de ellos se derivan:

- I. Nuestra principal prioridad es satisfacer al cliente a través de la **entrega temprana y continua de software de valor**.
- II. Son bienvenidos los requisitos cambiantes, incluso si llegan tarde al desarrollo. Los procesos ágiles se dobligan al cambio como ventaja competitiva para el cliente.
- III. Entregar con frecuencia software que funcione, en periodos de un par de semanas hasta un par de meses, con preferencia en los períodos breves.
- IV. Las **personas del negocio y los desarrolladores deben trabajar juntos** de forma cotidiana a través del proyecto.
- V. Construcción de proyectos en torno a individuos motivados, dándoles la oportunidad y el respaldo que necesitan y procurándoles confianza para que realicen la tarea
- VI. La forma más eficiente y efectiva de comunicar información de ida y vuelta dentro de un equipo de desarrollo es mediante la conversación cara a cara.
- VII. El **software que funciona es la principal medida del progreso**.
- VIII. Los procesos ágiles promueven el desarrollo sostenido. Los patrocinadores, desarrolladores y usuarios deben mantener un ritmo constante de forma indefinida.
- IX. La atención continua a la excelencia técnica enaltece la agilidad.
- X. La simplicidad como arte de maximizar la cantidad de trabajo que no se hace, es esencial.
- XI. Las mejores arquitecturas, requisitos y diseños emergen de equipos que se auto-organizan.
- XII. En intervalos regulares, el equipo reflexiona sobre la forma de ser más efectivo y ajusta su conducta en consecuencia.



Not like this....



1

2

3

4

Like this!



1

2

3

4

5



# “Iterar” no es lo mismo que “Incrementar”

1



2



3



Incrementar requiere tener una idea completa sobre el producto final

1



2



3



No hay una visión detallada del producto final, al contrario, el constante feedback recibido en cada entrega permite “ir descubriendo” el producto software.



*“Si con dos pizzas no puedes alimentar a tu equipo, no te falta pizza, te sobra equipo.”*

*Jeff Bezos, La ley de las Dos Pizzas*





*“Si bien puede usarse como analogía para la planificación de construcción de software la planificación de construcción de una casa o un rascacielos, difícilmente puede utilizarse [...] El software es más como la **jardinería**. Planifica el jardín, planta las plantas. Luego vea qué está creciendo, elimine las malas hierbas y plante nuevas plantas.”*

*Andrew Hunt, **The Pragmatic Programmer***





# Ciclo de vida del Software

---

*¿Qué es el Ciclo de Vida del Software?*

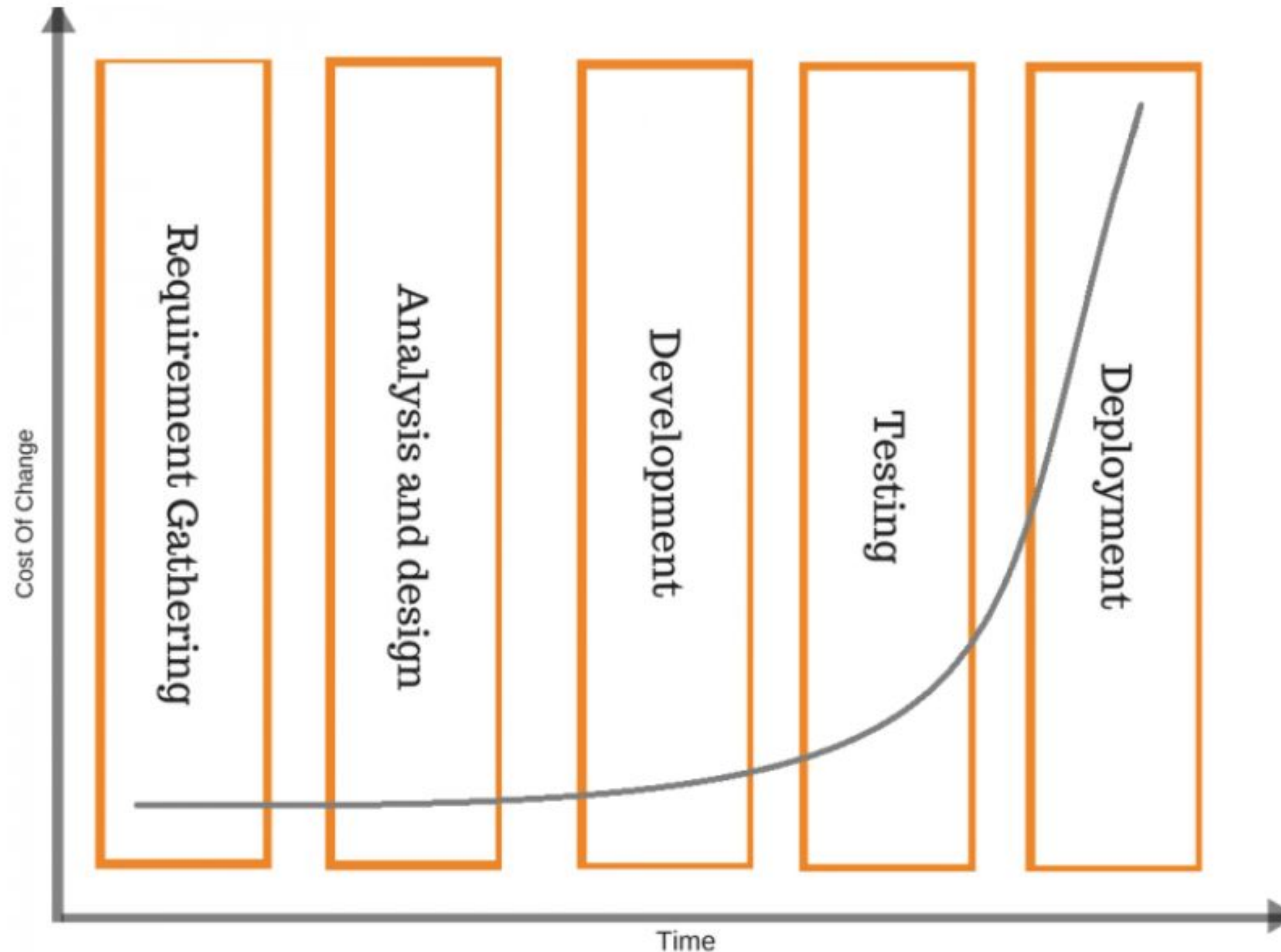
*Visión Predictiva*

*Visión Adaptativa*

*La curva del coste del cambio*



# LA CURVA DEL COSTE DEL CAMBIO



En Software Engineering Economics (Barry Boehm, 1981) se expone la siguiente gráfica para explicar que los cambios introducidos en etapas tardías de la construcción tienen un elevado coste.

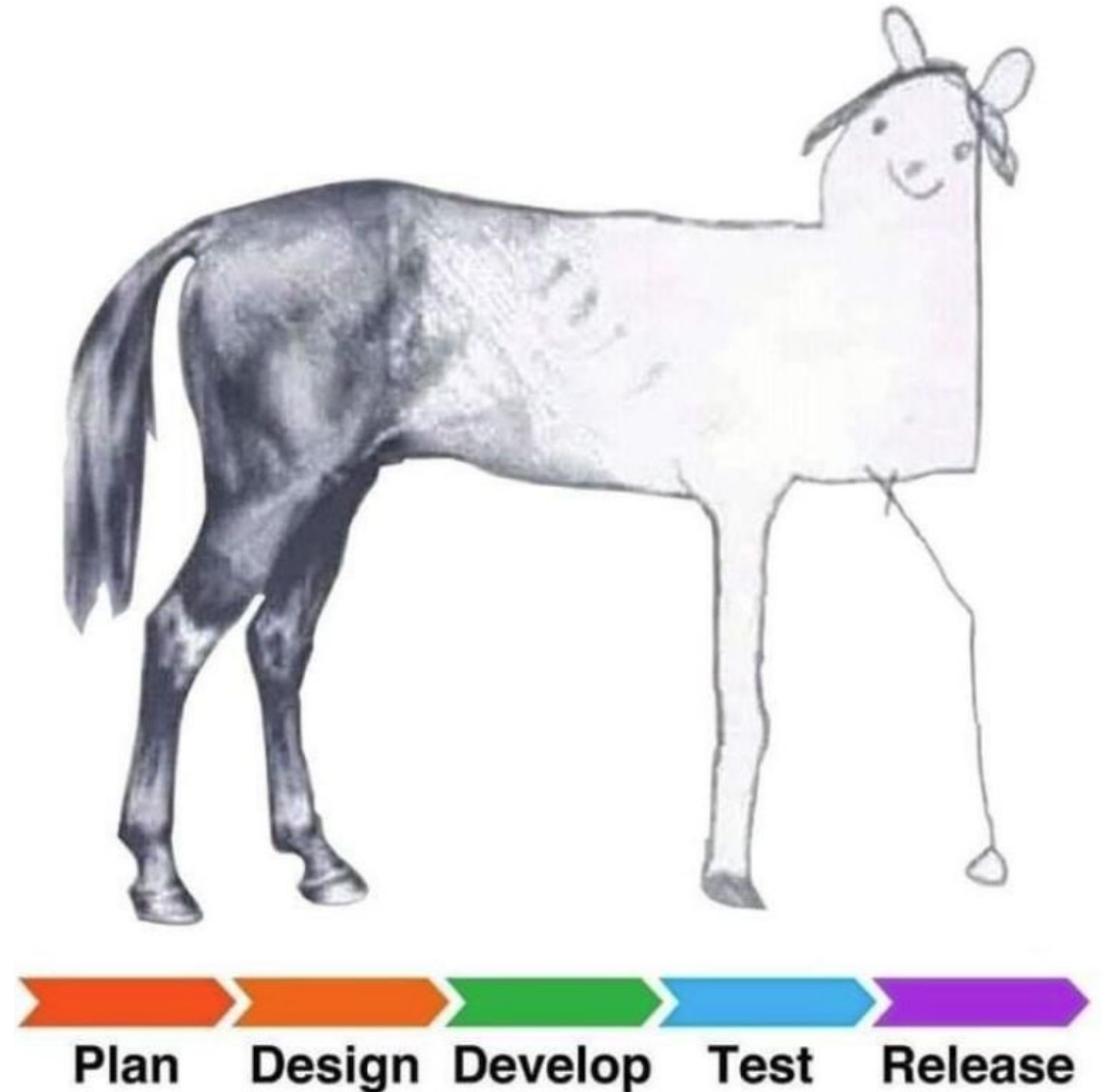




# LA CURVA DEL COSTE DEL CAMBIO

Este hecho fue usado durante los 80s y gran parte de los 90s para justificar los ciclos de vida en cascada de la visión predictiva. Despectivamente, los críticos de los 80s bautizaron BDUF (Big Design Up Front) a este tipo de enfoque. El BDUF se caracteriza por alargar enormemente las fases de Requerimientos y Análisis para elaborar pesados documentos de especificaciones.

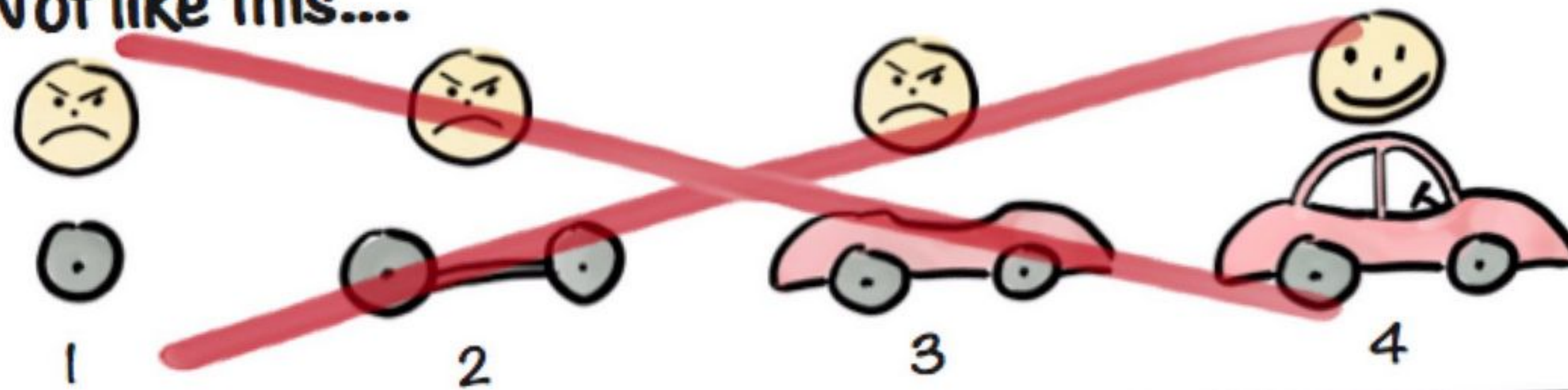
Las metodologías adaptativas huyen del “Gran Diseño Al Frente”, prefieren ir descubriendo el producto software en cada iteración, introduciendo cambios constantemente. Pero si los cambios introducidos en etapas tardías de la construcción son “caros”, ¿no deberían ser las metodologías adaptativas una ruina económica?



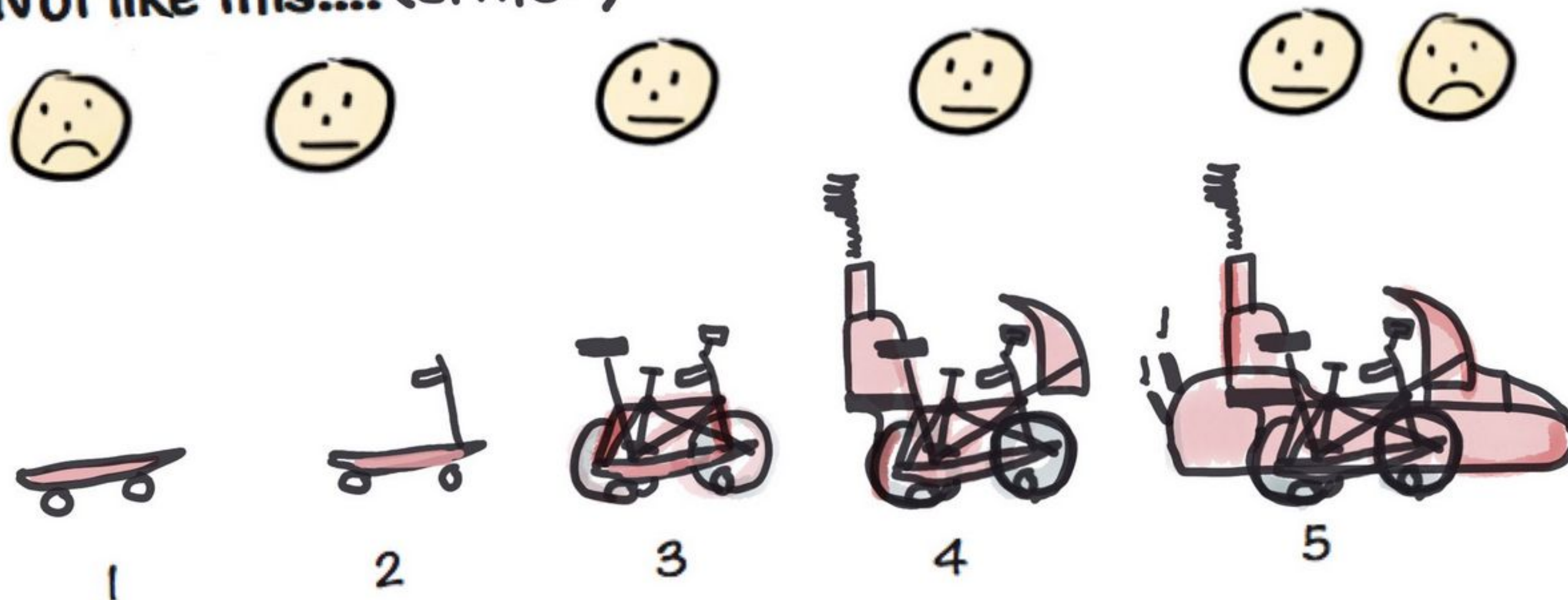


# LA CURVA DEL COSTE DEL CAMBIO

Not like this....



Not like this.... (either)



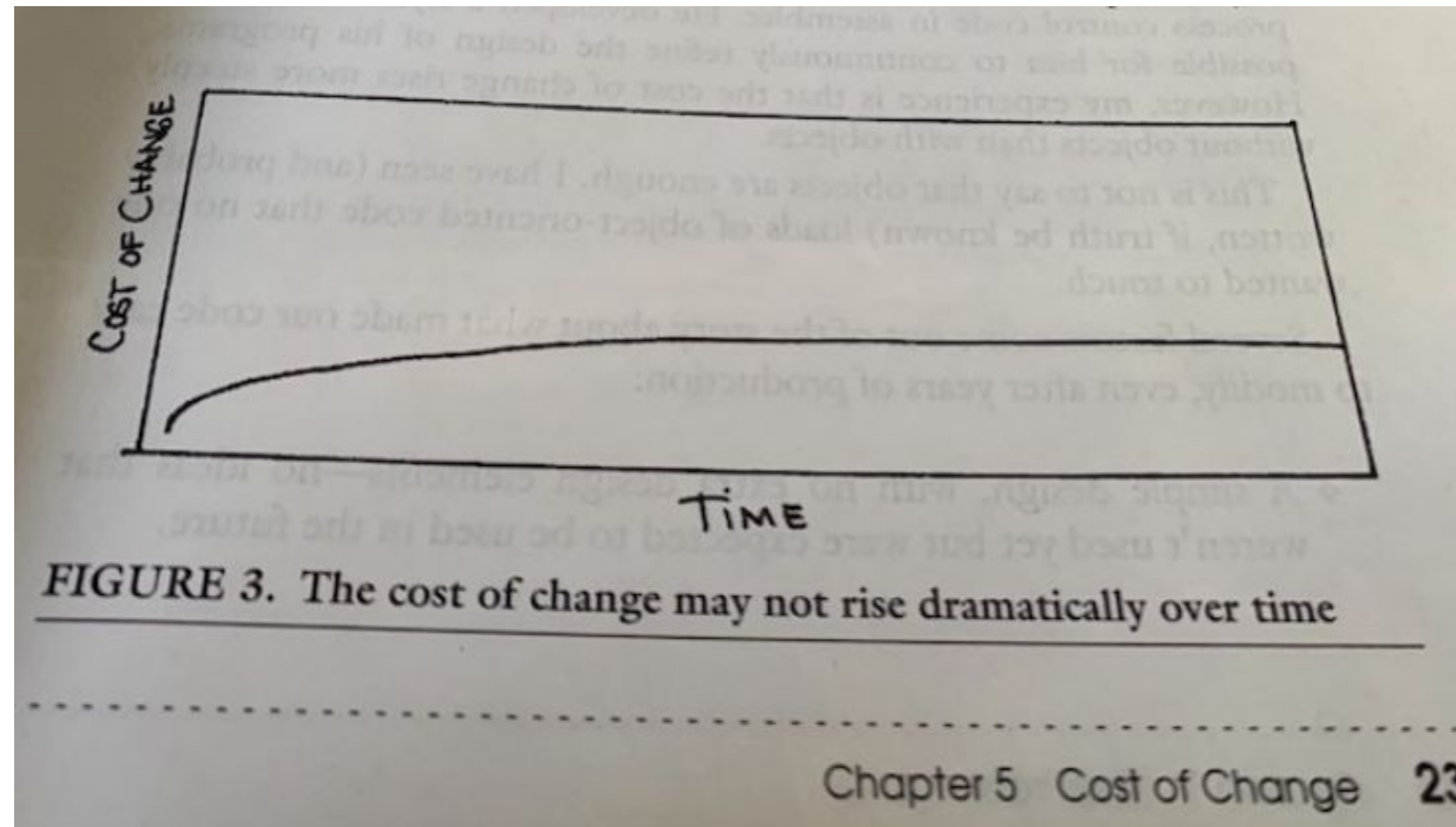
La solución más elemental pasaría por introducir cambios de la forma más barata reduciendo, por ejemplo, la calidad del producto software.

Reducir la calidad del producto software no es una buena idea: los errores y nuevas necesidades obligarán a pagar la factura con intereses. En la visión adaptativa es imprescindible abaratar el coste del cambio sin perjuicio de la calidad del producto software. ¿Existe alguna forma de reducir el coste del cambio en fases tardías de la construcción?



# LA CURVA DEL COSTE DEL CAMBIO

En “Extreme Programming” (1999) Kent Beck propone el uso de una técnica de codificación guiada por pruebas, TDD (Test Driven Development). Esta técnica de codificación logra aplanar la curva del coste de cambio.





# LEYES HEURÍSTICAS



Tanto la *ley de brooks* como la *matriz valor vs coste* como la *curva del coste del cambio* como el *radar de deuda técnica* como la *ley de las dos pizzas* como la *ley de Conway* son leyes heurísticas, han sido deducidas a partir del análisis de las mediciones y los datos empíricos (experimentales) obtenidos en la realización de estudios y proyectos.

Estas leyes heurísticas no surgieron de un modelo científico, sino de la experiencia. La formulación de estas leyes o pautas suponen el fundamento cualitativo para la propuesta de metodologías y prácticas utilizadas en la ingeniería del software.







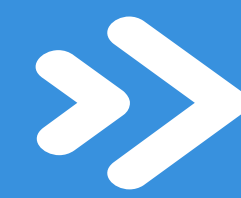
# TABLA DE CONTENIDOS

- 01** Breve Historia
- 02** ¿Qué es la Ingeniería del Software?
- 03** Ciclo de vida
- 04** Herramientas CASE
- 05** Diagramas



# Herramientas CASE

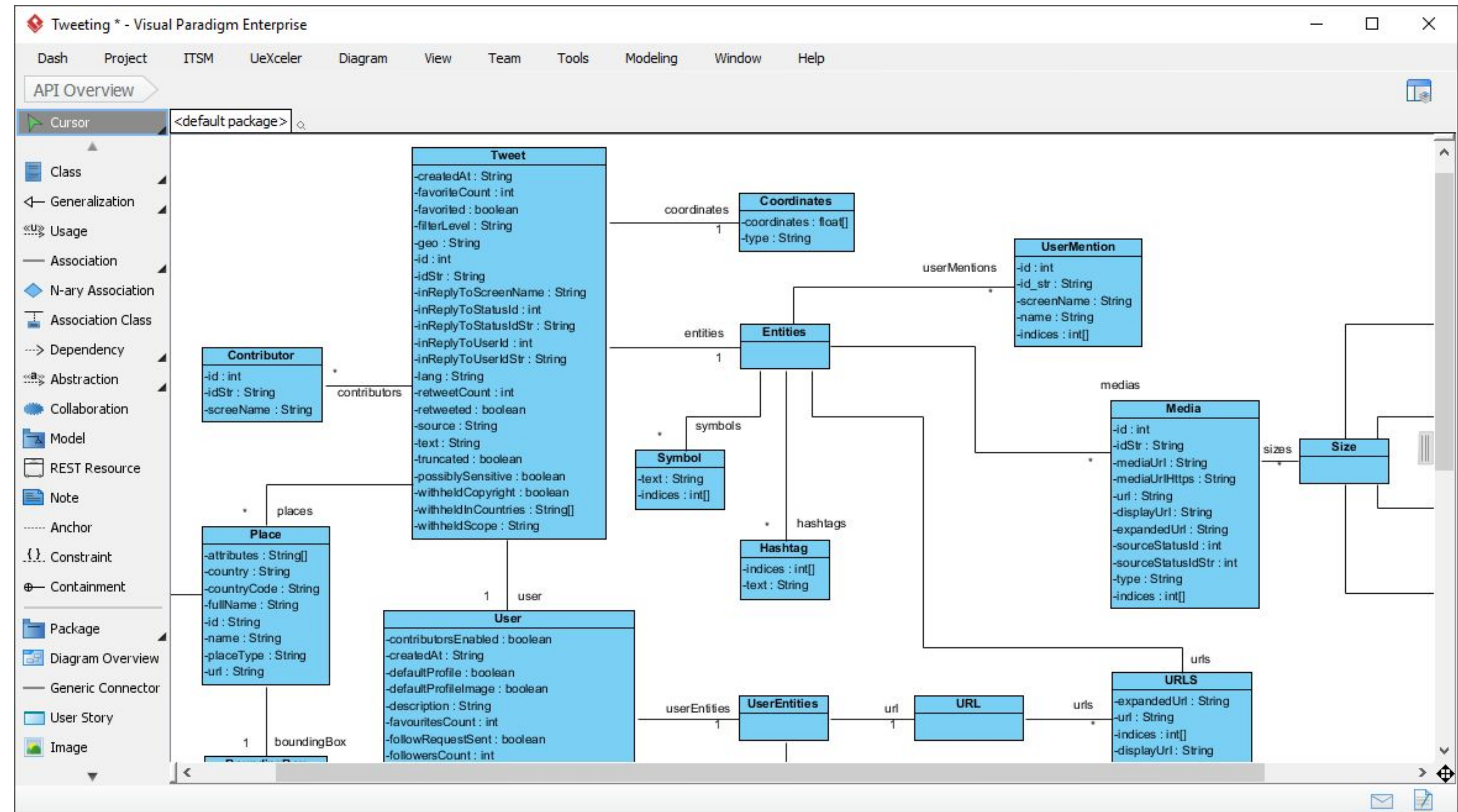




# Herramientas CASE

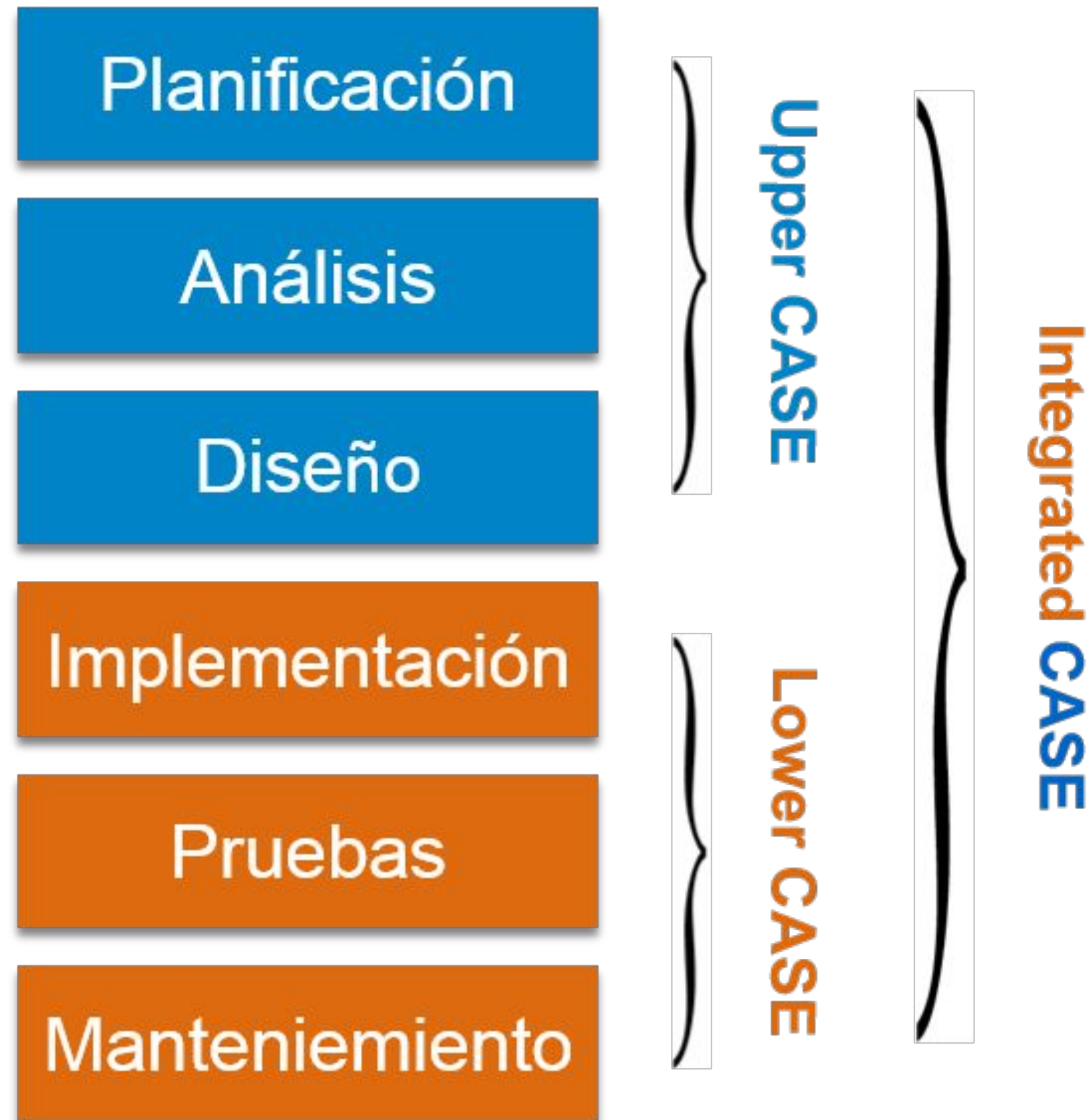
Las herramientas CASE (Computer Aided Software Engineering, Ingeniería de Software Asistida por Computadora) son aplicaciones informáticas destinadas a facilitar y automatizar la construcción de software.

Los diagramas UML exigen por parte del diseñador concreción y rigor, gracias a ello existen herramientas, denominadas CASE, capaces de traducir los diagramas en código (y el código en diagramas) de forma automática.





## >> Herramientas CASE



Existe un gran número de herramientas CASE. La clasificación de herramientas CASE más habitual se basa en las fases del ciclo de desarrollo que cubren:

- ❑ **Upper CASE**, herramientas que ayudan en las fases de planificación, análisis de requisitos y estrategia del desarrollo. Por ejemplo, Microsoft Project para la planificación del proyecto o Draw.io para el diseño.
- ❑ **Lower CASE**, herramientas que ayudan en las fases de implementación, pruebas y mantenimiento. Algunas de ellas son capaces de autogenerar código, detectan errores, soportan la depuración de programas, etc. Además automatizan la documentación completa de la aplicación. Por ejemplo, NetBeans.
- ❑ **Integrated CASE**, herramientas que son de utilidad en todas las fases del ciclo de vida. Por ejemplo, Visual Paradigm es una herramienta Middle CASE capaz de traducir diagramas UML en código y viceversa.



# TABLA DE CONTENIDOS

- 01** Breve Historia
- 02** ¿Qué es la Ingeniería del Software?
- 03** Ciclo de vida
- 04** Herramientas CASE
- 05** Diagramas



# Diagramas



## ¿Qué es un Diagrama?

Transmitir conceptos o ideas no es sencillo. Para una adecuada comprensión por parte del receptor resulta útil expresar lo mismo de diferentes maneras.

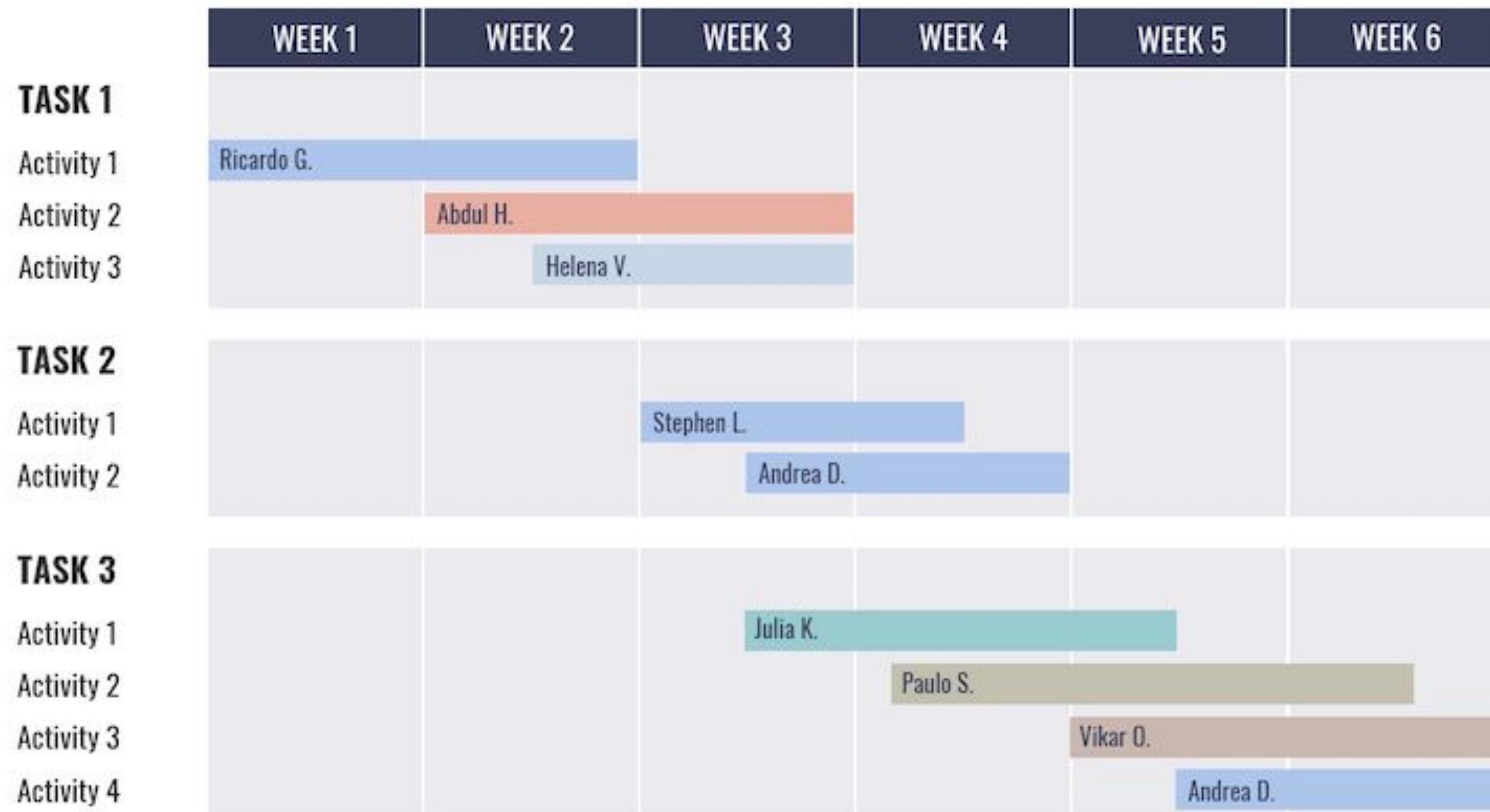
Por ejemplo, el término “*modelo descriptivo*” puede resultar excesivamente académico o abstracto para una persona con formación técnica básica, por ello complementarlo con el término “*archivo de configuración*”, más concreto, ayuda a asegurar que el mensaje es entendido correctamente.

Emplear diagramas puede facilitar la correcta comprensión de los sistemas, de la misma forma que emplear otras palabras ayuda a entender conceptos e ideas. Los diagramas hacen uso de un **lenguaje basado en dibujos para expresar un sistema desde otro punto de vista**. Así, los diagramas **permiten al receptor eliminar ambigüedades, despejar dudas, entendiendo unívocamente el sistema** que se pretende comunicar.



# ¿Qué es un Diagrama?

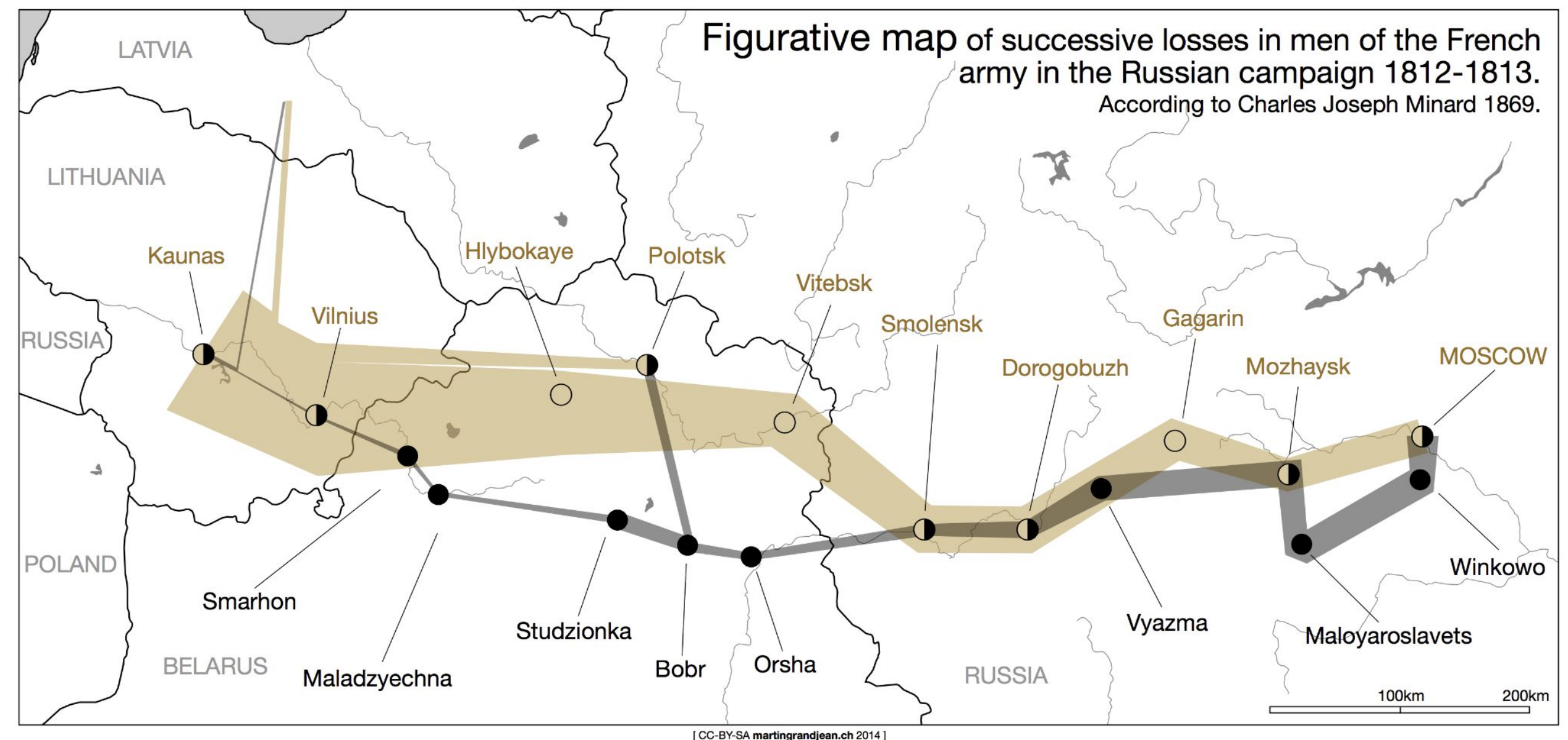
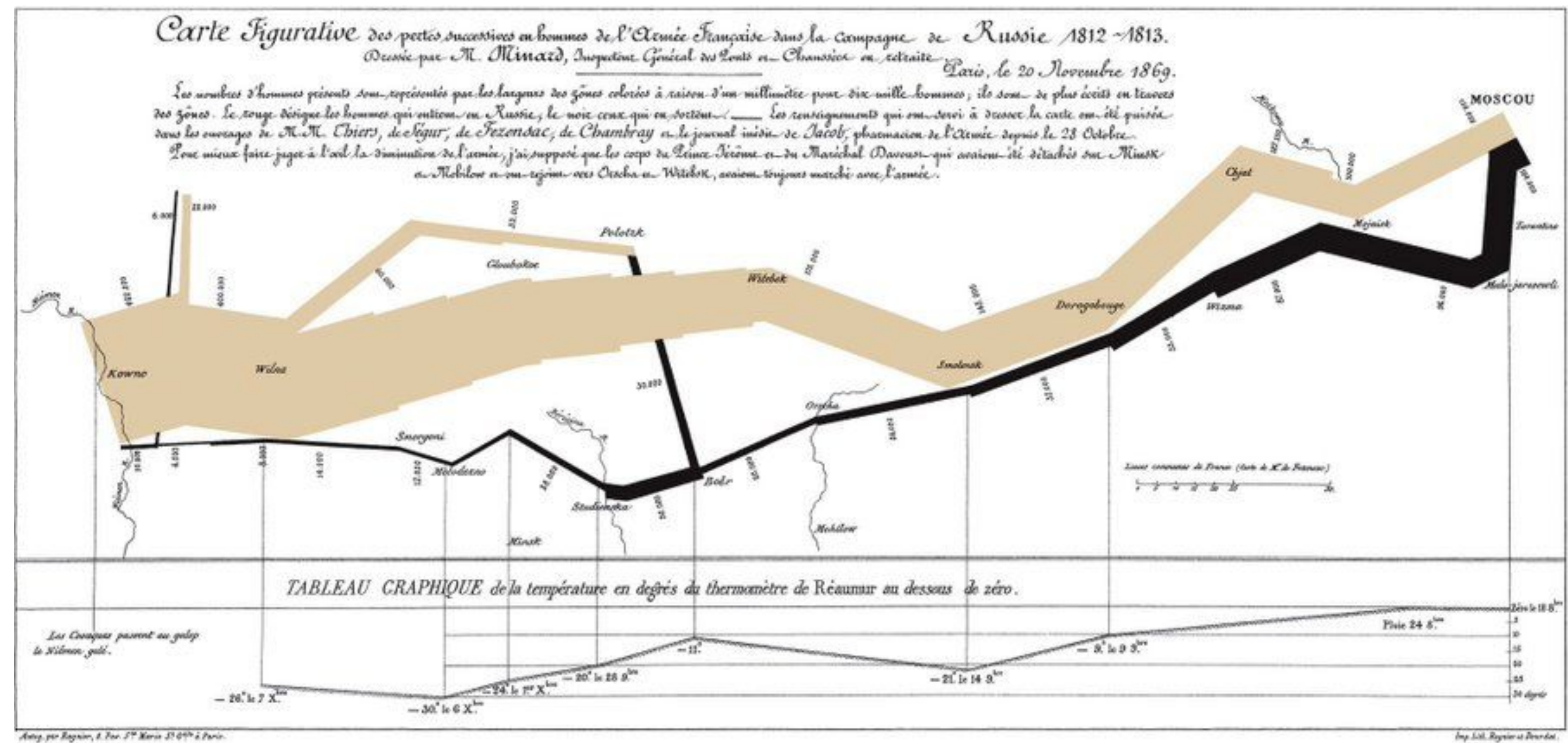
*“Un diagrama es una herramienta gráfica (dibujo) que ofrece una representación simplificada de un sistema o solución cuya función es transmitir visualmente la información que se desea resaltar de una manera directa y concisa.”*





# ¿Qué es un Diagrama?

¿Cómo explica un ingeniero la invasión napoleónica de 1812?  
¡Con un diagrama!





## ¿Qué es un Diagrama?

¿Cómo explica un ingeniero la planificación de un proyecto? ¡Con un diagrama!

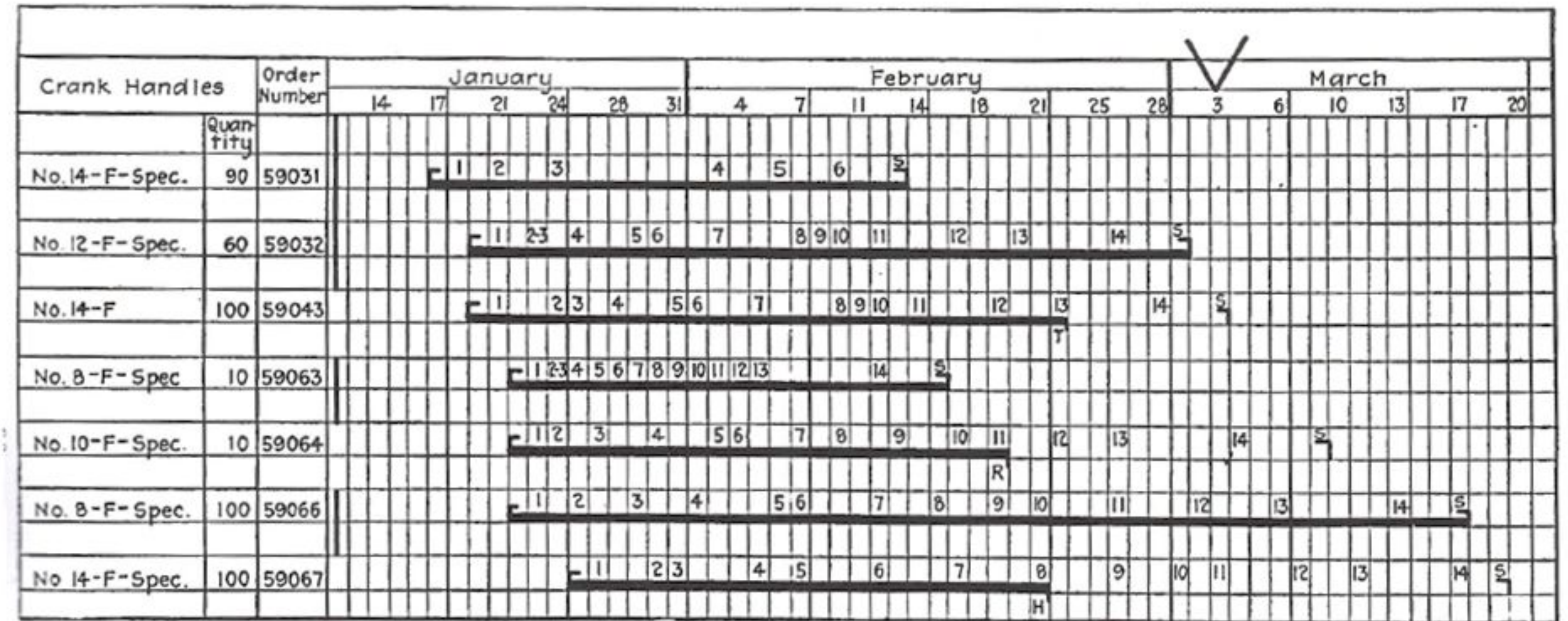


FIGURE 24. A GANTT PROGRESS CHART USED IN A PLANT WHICH MANUFACTURES ON ORDER

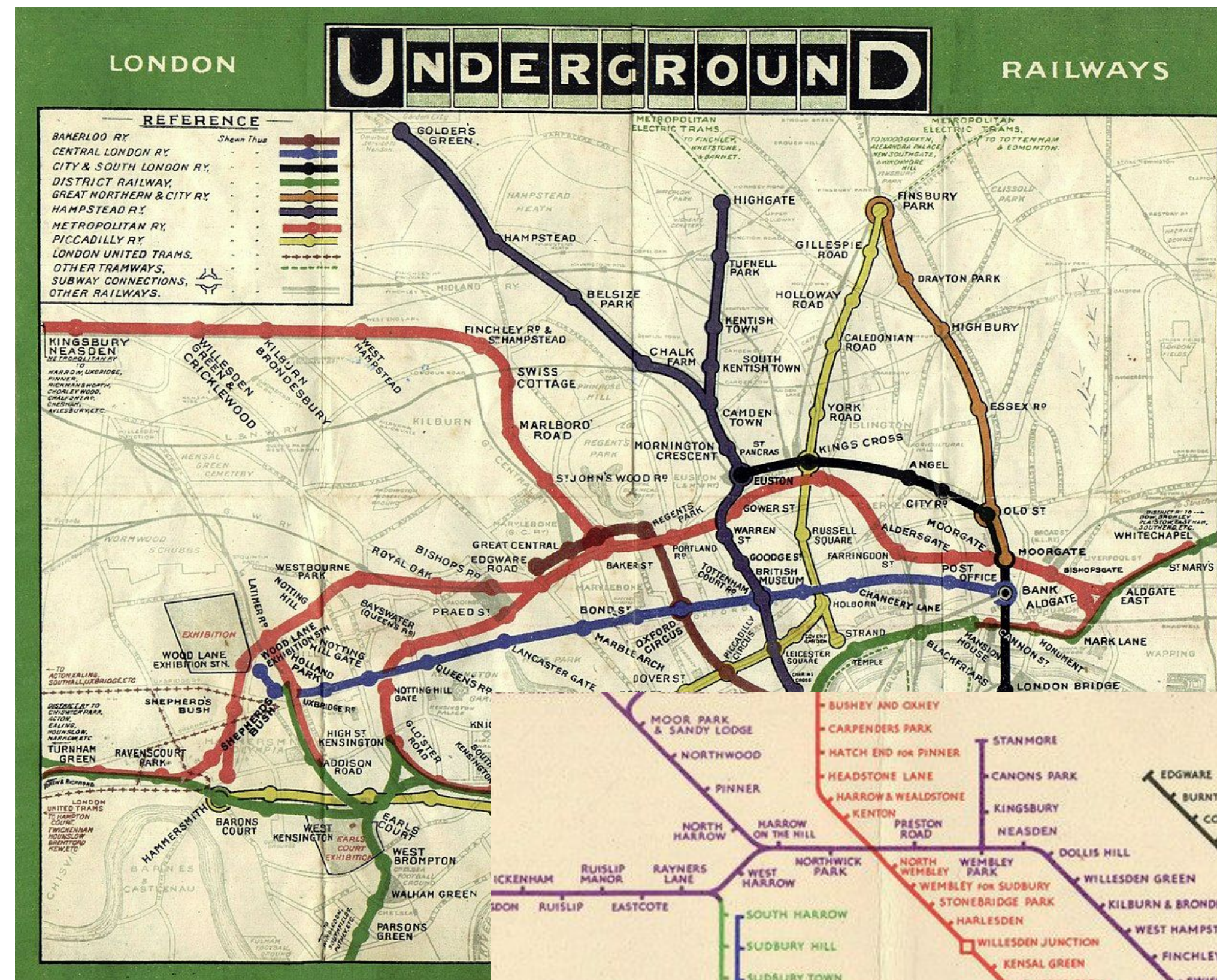
In this chart the angle opening to the right indicates the date on which the material was to be issued from stores; the figures indicate the operations to be done on the order and are placed under the dates on which they were to be begun; the angle opening to the left in-

diates the date on which the parts were to be shipped. The heavy lines show what operations have been done; and the letters under the lines indicate the reasons for delay. The *V* indicates that this chart was reproduced on March 3.



# ¿Qué es un Diagrama?

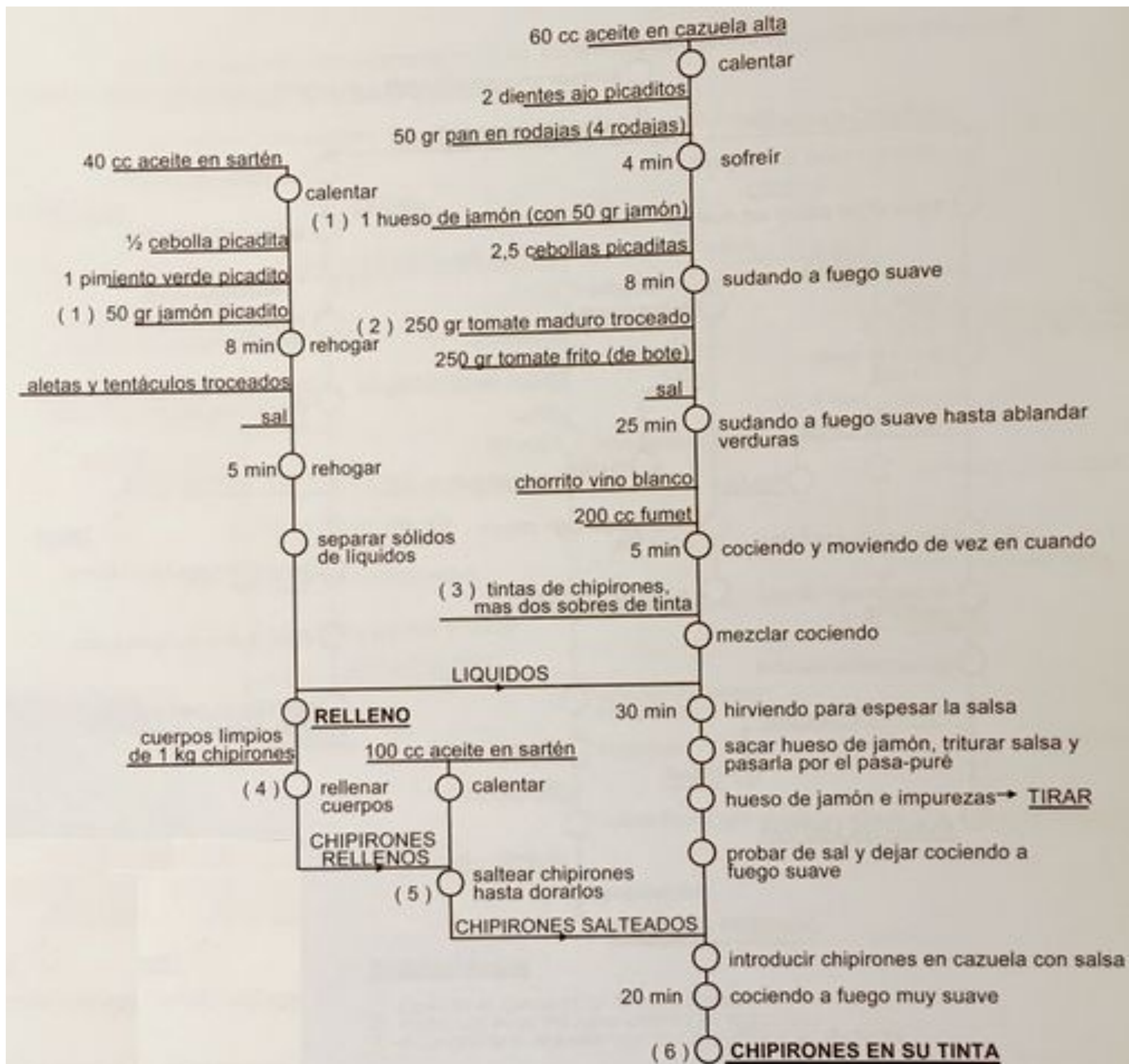
¿Cómo explica un ingeniero el metro de Londres? ¡Con un diagrama!





# ¿Qué es un Diagrama?

¿Cómo explica un ingeniero la receta de chipirones en su tinta?  
¡Con un diagrama!





## ¿Qué es un Diagrama?

De la misma forma que programar en un determinado lenguaje, por ejemplo Java, permite adquirir conocimientos básicos sobre el paradigma orientado a objetos perfectamente aplicables a otros lenguajes como C# o Python, dibujar determinados diagramas, por ejemplo diagramas de casos de uso o de actividades, permite adquirir conocimientos básicos sobre formalismos y comunicación gráfica perfectamente aplicables a otros tipos de diagramas como diagramas BPMN o [diagramas de modelado C4](#).



## Características de un buen diagrama

Dibujar un buen diagrama es tan difícil como redactar un buen párrafo o un buen resumen.

- **Son autoexplicativos y concisos, no son complicados**
- **Ayudan a entender la complejidad, no deben ser prescindibles**
- **Subrayan detalles, pero no tantos como para ocultar lo fundamental**
- **Son útiles, no tienen porqué ser normalizados**



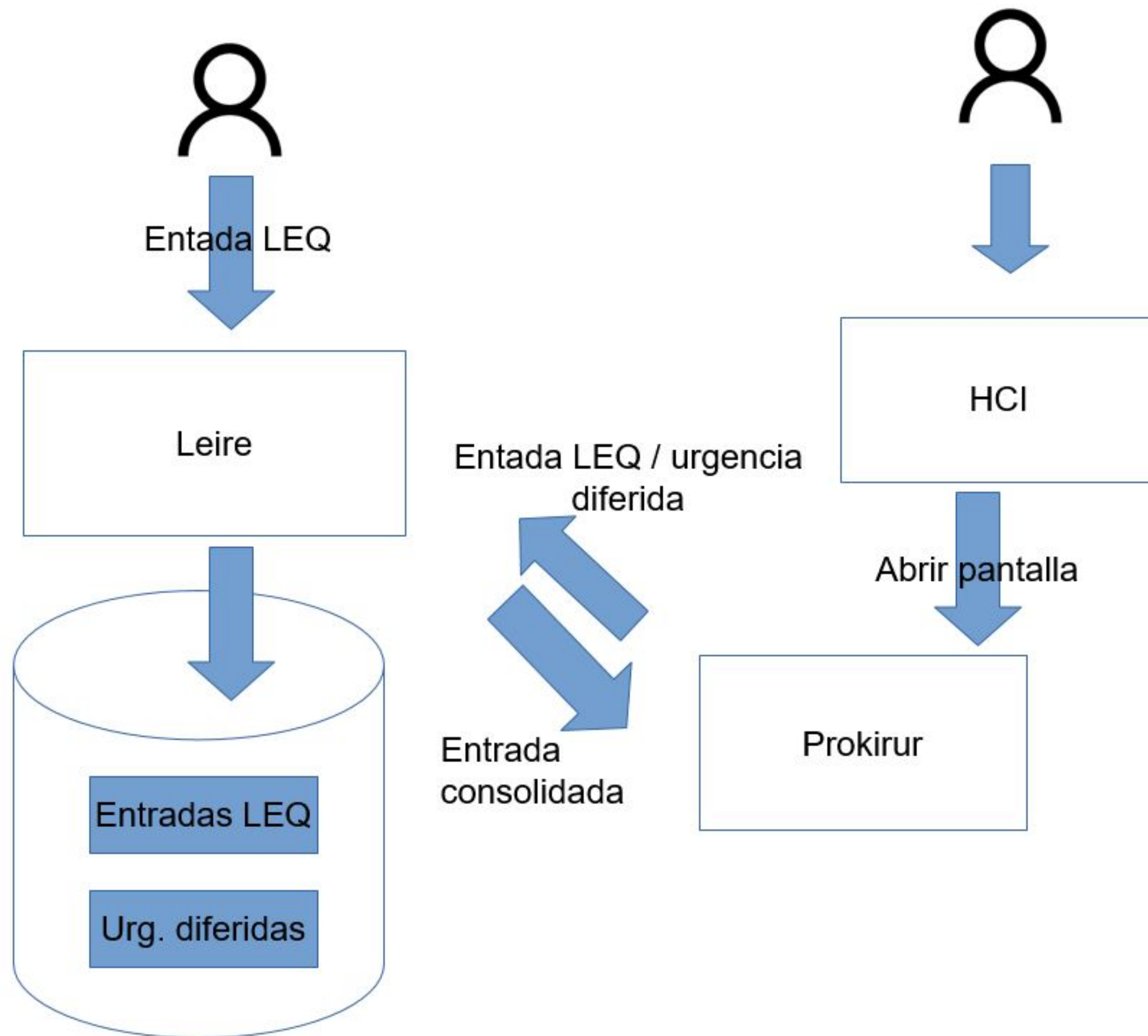
## Características de un buen diagrama

Dibujar un buen diagrama es tan difícil como redactar un buen párrafo o un buen resumen.

- **Son autoexplicativos y concisos, no son complicados**
- **Ayudan a entender la complejidad, no deben ser prescindibles**
- **Subrayan detalles, pero no tantos como para ocultar lo fundamental**
- **Son útiles, no tienen porqué ser normalizados**



## >> Son autoexplicativos y concisos, no son complicados



Este diagrama no es claro ni conciso:

- ❑ Las cajas “Leire”, “HCI” y “Prokirur” son productos software ¿A qué producto pertenece la base de datos que guarda “Entradas LEQ”?
- ❑ Las flechas parecen indicar un flujo de datos, por ejemplo una “entrada consolidada” viaja de “Leire” a “Prokirur”, entonces ¿que significa la flecha “Abrir pantalla”? ¿Las flechas simbolizan movimiento de datos o son acciones de usuario?
- ❑ Un usuario parece introducir en “Leire” una “Entrada LEQ”, ¿qué es lo que introduce el otro usuario en “HCI”?



## Características de un buen diagrama

Dibujar un buen diagrama es tan difícil como redactar un buen párrafo o un buen resumen.

- **Son autoexplicativos y concisos, no son complicados**
- **Ayudan a entender la complejidad, no deben ser prescindibles**
- **Subrayan detalles, pero no tantos como para ocultar lo fundamental**
- **Son útiles, no tienen porqué ser normalizados**



**>> Ayudan a entender la complejidad, no deben ser prescindibles**

## Concepto de solución



Para indicar que los datos del “analizador” se guardarán en “Infinity” no parece necesario dibujar un diagrama



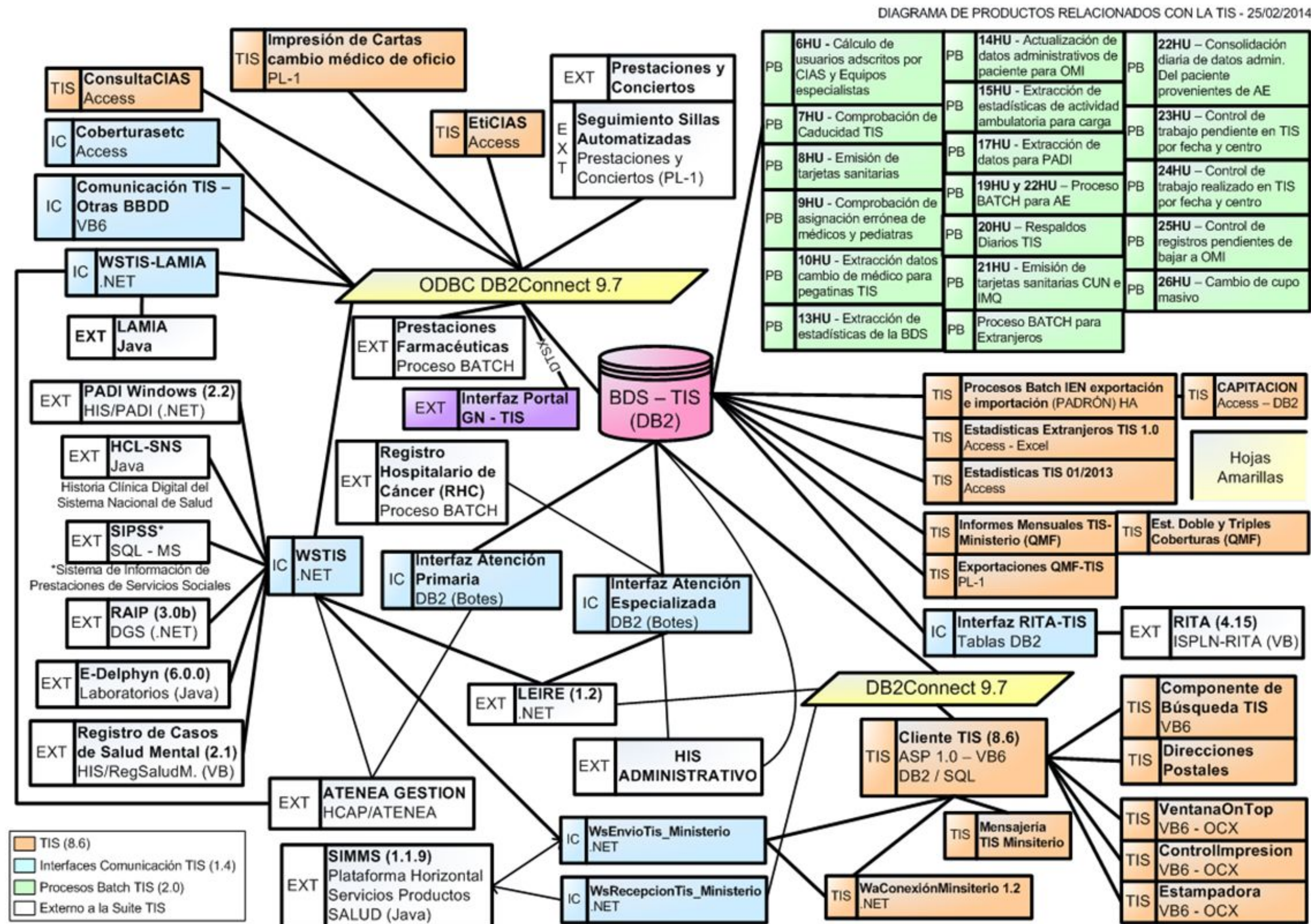
## Características de un buen diagrama

Dibujar un buen diagrama es tan difícil como redactar un buen párrafo o un buen resumen.

- Son autoexplicativos y concisos, **no son complicados**
- Ayudan a entender la complejidad, **no deben ser prescindibles**
- **Subrayan detalles, pero no tantos como para ocultar lo fundamental**
- Son útiles, **no tienen porqué ser normalizados**

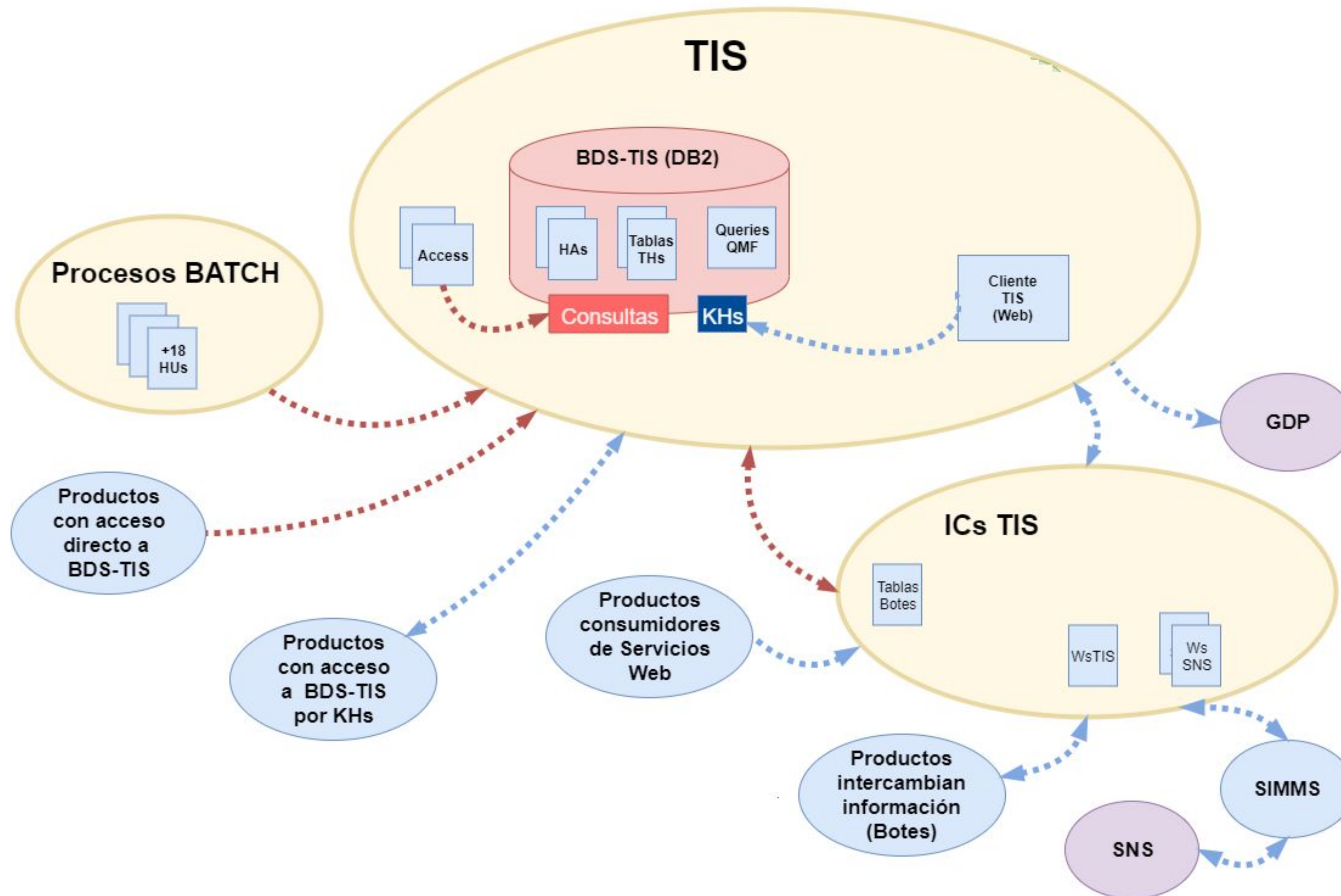


# >> Subrayan detalles, pero no tantos como para ocultar lo fundamental





➤➤ Subrayan detalles, pero no tantos como para ocultar lo fundamental



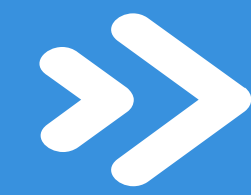


## Características de un buen diagrama

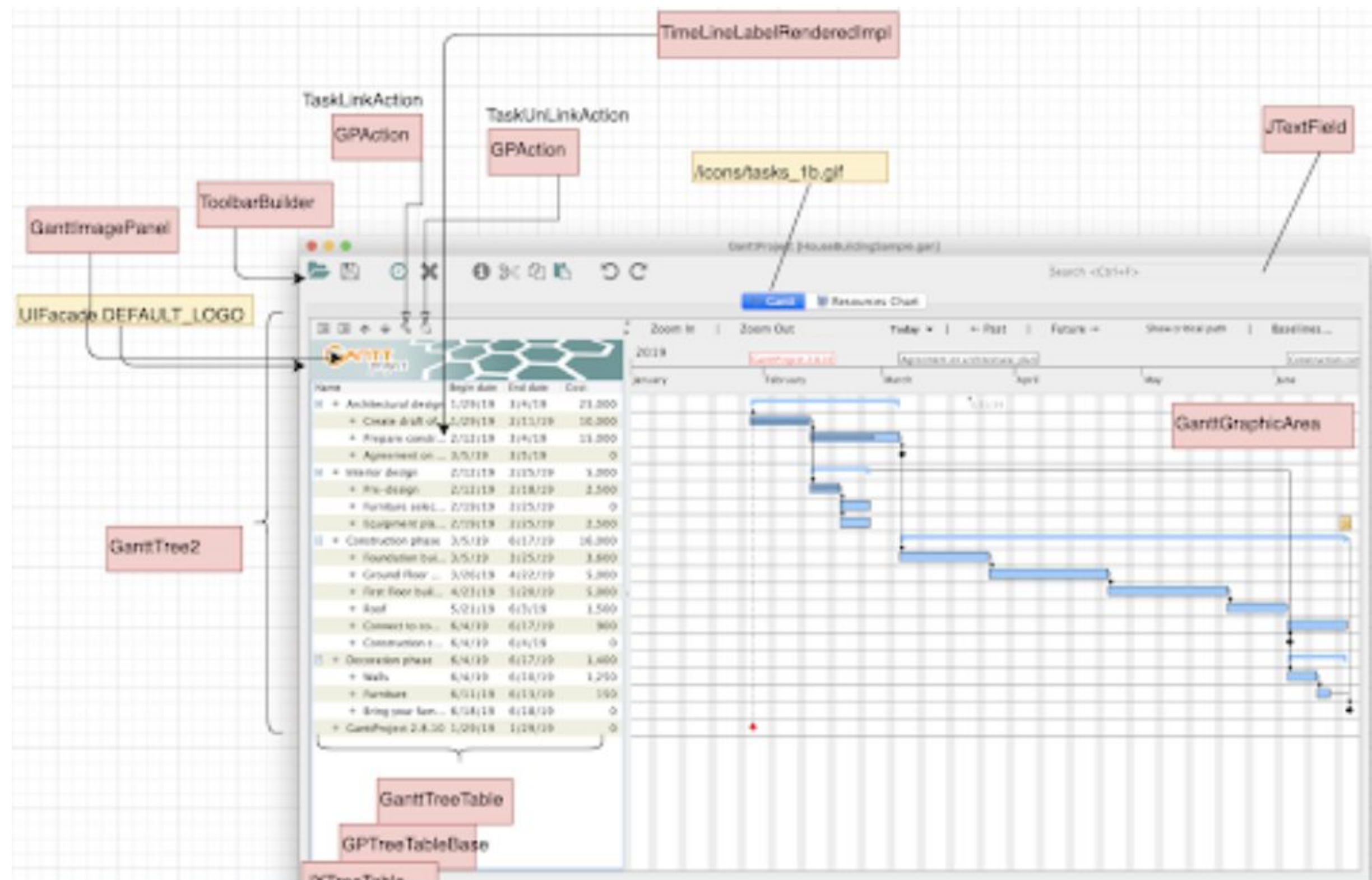
Dibujar un buen diagrama es tan difícil como redactar un buen párrafo o un buen resumen.

- **Son autoexplicativos y concisos, no son complicados**
- **Ayudan a entender la complejidad, no deben ser prescindibles**
- **Subrayan detalles, pero no tantos como para ocultar lo fundamental**
- **Son útiles, no tienen porqué ser normalizados**





Son útiles, no tienen porqué ser normalizados



Este diagrama muestra la relación entre el aspecto del interface de usuario y las principales clases responsables de su renderizado. No es un diagrama estándar (no tiene “nombre”) y, ciertamente, no parece muy riguroso, pero ello no impide que sea un buen diagrama capaz de comunicar información útil de forma rápida y concisa..



